

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG



BÁO CÁO ĐỒ ÁN MÔN HỌC

**ĐỀ TÀI: Deployment of Super Mario on Kubernetes
using Terraform**

Môn học: NT548.Q11 - Công nghệ DevOps và ứng dụng

Giảng viên hướng dẫn: Ths. Lê Anh Tuấn

Nhóm 19:

1. Nguyễn Văn Nam 22520922
2. Trần Ngọc Long 22520822
3. Phan Thị Bích Quyên 22521224

Thời gian thực hiện: 09/2025 – 12/2025

MỤC LỤC

MỤC LỤC.....	2
DANH SÁCH ẢNH.....	4
DANH SÁCH BẢNG.....	6
TÓM TẮT.....	7
CHƯƠNG 1: TỔNG QUAN.....	8
1.1. Giới thiệu đề tài.....	8
1.2. Mục tiêu đề tài.....	8
1.3. Phạm vi thực hiện.....	9
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	10
2.1. Tổng quan về DevOps.....	10
2.1.1. DevOps là gì.....	10
2.1.2. Tại sao áp dụng DevOps cho đồ án này.....	10
2.2. Infrastructure as Code (IaC) & Configuration Management.....	10
2.2.1. Infrastructure as Code là gì?.....	10
2.2.2. Terraform.....	11
2.3. Containerization & Orchestration.....	11
2.3.1. Docker & Docker Hub.....	11
2.3.2. Kubernetes (K8s).....	13
2.4. CI/CD (Continuous Integration / Continuous Deployment).....	14
2.4.1. CI/CD là gì?.....	14
2.4.2. GitHub Actions.....	14
2.5. GitOps với Argo CD.....	15
2.6. NGINX Ingress Controller.....	15
2.7. Giám sát hệ thống (Monitoring).....	16
2.7.1. Prometheus.....	16
2.7.2. Grafana.....	16
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	17
3.1. Kiến trúc hệ thống tổng quan.....	17
3.1.1. Kiến trúc môi trường Development.....	17
3.1.2. Kiến trúc môi trường Production.....	18
3.1.3. So sánh hai môi trường.....	20
3.2. Thiết kế quy trình DevOps.....	20
3.2.1. Chiến lược phân nhánh Git.....	20
3.2.2. Quy trình CI/CD cho môi trường Development.....	21
3.2.3. Quy trình CI/CD cho môi trường Production.....	22
3.2.4. GitOps với ArgoCD.....	23
CHƯƠNG 4: HIỆN THỰC VÀ TRIỂN KHAI.....	24
4.1. Chuẩn bị môi trường & Source code.....	24

Báo cáo Đồ án Công nghệ DevOps và Ứng dụng

4.2. Triển khai Hạ tầng với IaC (Terraform)	25
4.2.1. Cấu trúc thư mục	25
4.2.2. Module AKS	25
4.2.3. Module Network	26
4.2.4. Module Ingress	27
4.2.5. Cấu hình môi trường Development	27
4.2.6. Cấu hình môi trường Production	28
4.2.7. Orchestration với main.tf của môi trường	29
4.2.8. Quy trình triển khai	30
4.3. Đóng gói ứng dụng (Containerization)	31
4.3.1. Tổng quan về ứng dụng	31
4.3.2. Dockerfile cho Frontend	32
4.3.3. Dockerfile cho Backend	32
4.4. Xây dựng Pipeline CI/CD	33
4.4.1. Giai đoạn CI - Continuous Integration với GitHub Actions	33
4.4.2. Giai đoạn CD - Continuous Deployment với Argo CD	36
4.4.3. Helm Charts Structure	38
4.5. Triển khai Giám sát (Monitoring)	40
CHƯƠNG 5. KẾT QUẢ VÀ ĐÁNH GIÁ	42
5.1. Kiểm thử luồng CI/CD tự động	42
5.2. Đánh giá tổng quan	47
CHƯƠNG 6: KẾT LUẬN	48
6.1. Kết luận	48
6.2. Hạn chế	49
6.3. Hướng phát triển	49
TÀI LIỆU THAM KHẢO	50

DANH SÁCH ẢNH

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	10
Hình 2.1: Dockerfile cho backend game Mario.....	12
Hình 2.2: Kubernetes Ingress Architecture.....	16
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	17
Hình 3.1: Kiến trúc môi trường Development.....	17
Hình 3.2: Kiến trúc môi trường Production.....	18
Hình 3.3: Luồng hoạt động Development.....	21
Hình 3.4: Luồng hoạt động Production.....	22
CHƯƠNG 4: HIỆN THỰC VÀ TRIỂN KHAI	24
Hình 4.1: Cấu trúc thư mục của Repo	24
Hình 4.2: Cấu trúc thư mục terraform.....	25
Hình 4.3: Code file main.tf của module AKS	26
Hình 4.4: Code file main.tf của module network.....	26
Hình 4.5: Code file main.tf của module ingress.....	27
Hình 4.6: Code file .tfvars của môi trường development	28
Hình 4.7: Code file .tfvars của môi trường production	28
Hình 4.8: Code file main.tf của môi trường development (1)	29
Hình 4.9: Code file main.tf của môi trường development (2)	30
Hình 4.10: Các lệnh để deploy infrastructure môi trường development.....	30
Hình 4.11: Frontend thực hiện gọi API tới Backend	31
Hình 4.12: File backend app.py	31
Hình 4.13: Dockerfile cho Frontend.....	32
Hình 4.14: Dockerfile cho Backend	32
Hình 4.15: Danh sách các CI workflows.....	33
Hình 4.16: Điều kiện trigger workflows CI - Dev Frontend	34
Hình 4.17: Đăng nhập vào Docker Hub và build Image.....	34
Hình 4.18: Scan Image đã build với Trivy.....	34
Hình 4.19: Push Image và cập nhật helm value tương ứng.....	35
Hình 4.20: Danh sách các Manifest ArgoCD Applications	36
Hình 4.21: Manifest Argo CD Application cho frontend ở môi trường development.....	37
Hình 4.22: Nội dung argocd-ingress-prod.yaml.....	38
Hình 4.23: Cấu trúc thư mục helm.....	38
Hình 4.24: Value phần deployment trong file value-dev frontend (1).....	39
Hình 4.25: Value phần deployment trong file value-dev frontend (2).....	39
Hình 4.26: Value phần service trong file value-dev frontend	39
Hình 4.26: Value phần ingress trong file value-dev frontend	40
Hình 4.27: Cài đặt Prometheus và Grafana	40
CHƯƠNG 5. KẾT QUẢ VÀ ĐÁNH GIÁ.....	42

Báo cáo Đồ án Công nghệ DevOps và Ứng dụng

Hình 5.1: Commit và push cập nhật backend ở nhánh develop.....	42
Hình 5.2: CI - dev Backend được trigger.....	43
Hình 5.3: Trên Docker Hub có tag dev mới.....	43
Hình 5.4: ArgoCD refresh và sync thành công.....	43
Hình 5.5: Backend môi trường development đã được cập nhật.....	44
Hình 5.6: Tạo pull request và merge cập nhật từ develop vào main.....	44
Hình 5.7: Tạo tag và push.....	45
Hình 5.8: CI - Release Backend được trigger.....	45
Hình 5.9: Trên Docker Hub có Tag version	45
Hình 5.10: ArgoCD refresh và Sync thành công.....	46
Hình 5.11: Deploy Backend Production mới thành công.....	46

DANH SÁCH BẢNG

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	17
<i>Bảng 3.1: Cấu hình môi trường Development.....</i>	<i>17</i>
<i>Bảng 3.2: Cấu hình môi trường Production.....</i>	<i>19</i>
<i>Bảng 3.3: So sánh 2 môi trường.....</i>	<i>20</i>
CHƯƠNG 4: HIỆN THỰC VÀ TRIỂN KHAI.....	24
<i>Bảng 4.1: So sánh Dev và Release workflows.....</i>	<i>36</i>
<i>Bảng 4.2: So sánh value của backend và frontend môi trường production</i>	<i>40</i>
<i>Bảng 4.3: So sánh value của backend và frontend môi trường development</i>	<i>40</i>

TÓM TẮT

Trong quá trình phát triển phần mềm, việc triển khai thủ công thường mất nhiều thời gian, dễ xảy ra lỗi và khó đảm bảo tính đồng bộ giữa các môi trường. Với các ứng dụng cần cập nhật thường xuyên như game hoặc hệ thống web, cách triển khai này không còn phù hợp.

Trong đồ án này, nhóm xây dựng hệ thống triển khai tự động cho ứng dụng Mario theo hướng DevOps. Ứng dụng được đóng gói bằng Docker, quy trình CI được thực hiện thông qua GitHub Actions và push image lên Docker Hub. Hạ tầng được xây dựng bằng Terraform với hai AKS clusters riêng biệt cho dev và prod, ứng dụng được triển khai theo mô hình GitOps với Argo CD.

Kết quả đạt được là toàn bộ quá trình build và deploy ứng dụng Mario được tự động hóa, giảm thời gian triển khai, hạn chế lỗi thủ công và giúp hệ thống dễ quản lý, mở rộng trong tương lai.

CHƯƠNG 1: TỔNG QUAN

1.1. Giới thiệu đề tài

Trong phát triển phần mềm hiện đại, việc triển khai thủ công gây ra nhiều vấn đề: mất thời gian, dễ sai sót, và khó đảm bảo đồng bộ giữa các môi trường. Với các ứng dụng cần cập nhật liên tục như game hay web app, phương pháp này không còn hiệu quả.

DevOps ra đời để giải quyết những vấn đề trên bằng cách tự động hóa quy trình từ phát triển đến triển khai. Các công cụ như Docker, Kubernetes, và CI/CD pipeline giúp đội ngũ phát triển đưa sản phẩm lên production nhanh chóng và đáng tin cậy hơn. Đồ án này xây dựng hệ thống triển khai tự động cho game Mario, áp dụng các công nghệ DevOps phổ biến.

Ứng dụng được đóng gói bằng Docker, hạ tầng được quản lý bằng Terraform, CI/CD được thực hiện qua GitHub Actions, và triển khai lên Azure Kubernetes Service theo mô hình GitOps với Argo CD. Mục đích của đồ án là minh họa quy trình DevOps hoàn chỉnh trong thực tế, từ commit code đến ứng dụng chạy trên production mà không cần can thiệp thủ công.

1.2. Mục tiêu đề tài

Mục tiêu chính là xây dựng một hệ thống triển khai tự động hoàn chỉnh cho ứng dụng Mario, bao gồm:

- Về kỹ thuật: Containerize ứng dụng bằng Docker, xây dựng CI/CD pipeline với GitHub Actions, triển khai infrastructure bằng Terraform, chạy ứng dụng trên AKS và sử dụng Argo CD cho GitOps workflow. Ngoài ra cần thiết lập monitoring cơ bản để theo dõi hệ thống.
- Về quy trình: Tự động hóa toàn bộ quá trình từ code đến production, giảm thời gian triển khai và hạn chế lỗi thủ công. Đảm bảo tính nhất quán giữa các môi trường và tạo nền tảng dễ mở rộng sau này.
- Về học tập: Nắm vững các công cụ DevOps trong thực tế, hiểu rõ quy trình CI/CD và GitOps, có kinh nghiệm làm việc với Kubernetes và cloud platform.

1.3. Phạm vi thực hiện

Đồ án triển khai ứng dụng game Mario (web-based) lên Azure Kubernetes Service với hai clusters riêng biệt cho môi trường dev và prod. Toàn bộ infrastructure được quản lý bằng Terraform, bao gồm hai AKS clusters, Azure Container Registry, networking và security configurations.

CI pipeline sử dụng GitHub Actions để tự động build và push Docker image khi có commit mới trên các branch tương ứng. CD pipeline dùng Argo CD để deploy ứng dụng theo mô hình GitOps, tự động sync từ Git repository lên từng môi trường. Traffic vào ứng dụng được quản lý bởi NGINX Ingress Controller, giúp routing request và expose service ra ngoài với domain riêng cho mỗi môi trường.

Hệ thống cũng có monitoring cơ bản với Azure Monitor hoặc Prometheus/Grafana để theo dõi trạng thái cluster, pods và application metrics.

Đồ án không phát triển game từ đầu mà sử dụng source code có sẵn. Không triển khai multi-region, disaster recovery hay bảo mật nâng cao như penetration testing, vulnerability scanning chi tiết. Không thực hiện performance testing và load testing quy mô lớn. Không thiết lập complete observability stack với distributed tracing và APM chi tiết.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về DevOps

2.1.1. DevOps là gì

DevOps là sự kết hợp giữa Development (phát triển) và Operations (vận hành), nhằm rút ngắn vòng đời phát triển phần mềm và cung cấp sản phẩm chất lượng cao một cách liên tục. Trước đây, team dev và ops thường làm việc tách biệt, dẫn đến việc code chạy tốt trên máy dev nhưng gặp lỗi khi lên production, hoặc quá trình deploy kéo dài vì phải chờ ops xử lý thủ công.

DevOps không chỉ là công cụ hay quy trình, mà còn là văn hóa làm việc. Mô hình CAMS thể hiện các giá trị cốt lõi: Culture (văn hóa hợp tác giữa các team), Automation (tự động hóa các tác vụ lặp đi lặp lại), Measurement (đo lường hiệu suất để cải thiện liên tục), và Sharing (chia sẻ kiến thức và trách nhiệm).

2.1.2. Tại sao áp dụng DevOps cho đồ án này

Với ứng dụng game Mario, việc áp dụng DevOps giúp team có thể update tính năng mới hoặc fix bug nhanh chóng mà không lo ảnh hưởng đến người dùng. Mỗi khi có code mới, hệ thống tự động kiểm tra, build và deploy lên môi trường dev trước, sau đó lên prod một cách kiểm soát. Điều này đặc biệt quan trọng với game, nơi mà downtime hay lỗi có thể làm mất trải nghiệm người chơi.

2.2. Infrastructure as Code (IaC) & Configuration Management

2.2.1. Infrastructure as Code là gì?

Trước đây, việc setup server, network, storage được thực hiện thủ công qua giao diện web hoặc CLI. Cách này gặp vấn đề khi cần tái tạo môi trường: dễ quên các bước, cấu hình khác nhau giữa dev và prod, và không thể version control infrastructure như code.

IaC giải quyết vấn đề này bằng cách mô tả infrastructure dưới dạng code. Bạn khai báo "tôi cần 1 AKS cluster với 3 nodes, RAM 8GB mỗi node" trong file config, sau đó tool tự động tạo ra đúng như vậy. Nếu cần thay đổi, chỉ cần sửa file và apply lại.

Infrastructure trở nên reproducible, versionable và có thể review như code thông thường.

2.2.2. Terraform

Terraform là tool IaC phổ biến nhất hiện nay, hỗ trợ nhiều cloud provider như AWS, Azure, GCP. Đồ án chọn Terraform vì nó declarative (khai báo trạng thái mong muốn thay vì viết từng bước), có cộng đồng lớn với nhiều module sẵn, và quản lý state tốt.

Các khái niệm chính:

- State file: Terraform lưu trạng thái hiện tại của infrastructure trong file `.tfstate`. File này ghi lại những gì đã được tạo ra (IDs, IP addresses, etc.) để Terraform biết cần thay đổi gì khi bạn update config. State file nên được lưu remote (Azure Storage, S3) thay vì local để team có thể làm việc chung.
- Lifecycle: Quy trình làm việc với Terraform gồm ba bước chính. `terraform init` khởi tạo working directory, download providers cần thiết. `terraform plan` so sánh config với state hiện tại, cho biết sẽ tạo/sửa/xóa gì mà không thực sự thay đổi. `terraform apply` thực thi những thay đổi đó lên infrastructure thật. Cách làm này giúp kiểm soát tốt, tránh thay đổi nhầm.

Với đồ án này, Terraform giúp tạo toàn bộ AKS cluster, networking, ACR một cách nhất quán. Nếu cần rebuild hoặc tạo thêm môi trường mới, chỉ cần chạy lại Terraform thay vì click thủ công hàng chục bước trên Azure Portal.

2.3. Containerization & Orchestration

2.3.1. Docker & Docker Hub

Docker giải quyết vấn đề "it works on my machine" bằng cách đóng gói ứng dụng cùng tất cả dependencies vào một container. Container giống như một "hộp kín" chứa code, runtime, libraries, đảm bảo ứng dụng chạy giống nhau ở mọi nơi.

Các khái niệm chính:

Báo cáo Đồ án Công nghệ DevOps và Ứng dụng

- Image: Là template chứa filesystem và metadata để tạo container. Ví dụ image nginx:latest chứa web server nginx và các file cần thiết. Image được build từ Dockerfile và có thể push lên registry để share.
- Container: Là instance đang chạy của image. Một image có thể tạo nhiều containers. Container isolated với host system, có filesystem, network và process riêng.
- Dockerfile: File text chứa các lệnh để build image. Ví dụ với backend ứng dụng Mario, Dockerfile này tạo container chạy ứng dụng Python (FastAPI) trên Python 3.11, cài đặt các dependencies, copy code vào container, mở cổng 8000 và khởi chạy server Uvicorn để phục vụ ứng dụng.

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Hình 2.1: Dockerfile cho backend game Mario

Docker Hub cung cấp public repositories miễn phí (ai cũng có thể pull) và private repositories có giới hạn. Nó cũng tích hợp với GitHub để tự động build image khi có code mới, nhưng đồ án sử dụng GitHub Actions để kiểm soát CI pipeline tốt hơn.

Tại sao dùng Docker: Ứng dụng Mario được container hóa, đảm bảo chạy nhất quán trên mọi môi trường, tránh lỗi do khác phiên bản phần mềm giữa dev và production.

2.3.2. Kubernetes (K8s)

Khi có hàng chục, hàng trăm containers, việc quản lý thủ công trở nên không khả thi. Kubernetes là hệ thống orchestration giúp tự động deploy, scale và quản lý containerized applications.

Kubernetes cluster gồm hai loại nodes:

- Control Plane (Master Nodes): Quản lý toàn bộ cluster. Các thành phần chính gồm API Server (nhận requests từ kubectl), Scheduler (quyết định pod chạy trên node nào), Controller Manager (đảm bảo trạng thái mong muốn được duy trì), và etcd (database lưu config và state).
- Worker Nodes: Chạy các ứng dụng thực sự. Mỗi worker có Kubelet (agent giao tiếp với control plane), Container Runtime (Docker hoặc containerd để chạy containers), và Kube-proxy (quản lý network rules).

Các thành phần quan trọng:

- Pod: Đơn vị nhỏ nhất trong K8s, chứa một hoặc nhiều containers chạy cùng nhau. Ví dụ ứng dụng Mario chạy trong một pod với nginx container. Pods có lifecycle ngắn, có thể bị xóa và tạo lại bất cứ lúc nào.
- Deployment: Quản lý việc tạo và update pods. Bạn khai báo "tôi muốn 3 replicas của Mario app", Deployment sẽ đảm bảo luôn có đúng 3 pods chạy. Nếu 1 pod die, nó tự động tạo pod mới thay thế.
- Service: Cung cấp network endpoint ổn định cho pods. Vì pods có thể bị tạo/xóa với IP thay đổi liên tục, Service tạo một virtual IP không đổi và load balance traffic đến các pods phía sau.
- Ingress: Quản lý external access vào services trong cluster. Thay vì expose từng service ra ngoài, Ingress cho phép định nghĩa routing rules.

Tại sao dùng Kubernetes: Với đồ án này, K8s giúp quản lý hai môi trường dev và prod độc lập trên hai clusters riêng biệt, đảm bảo môi trường prod hoàn toàn tách biệt và không bị ảnh hưởng bởi testing trên dev. K8s tự động restart app nếu crash và để

dàng scale khi cần. AKS (Azure Kubernetes Service) là managed Kubernetes, Azure lo phần control plane nên ta chỉ cần tập trung vào deploy app.

2.4. CI/CD (Continuous Integration / Continuous Deployment)

2.4.1. CI/CD là gì?

Continuous Integration (CI): Là practice mà developers merge code vào main branch thường xuyên (nhiều lần mỗi ngày). Mỗi lần merge, hệ thống tự động build và test code để phát hiện lỗi sớm. Thay vì code 1 tháng mới merge và phát hiện conflict khổng lồ, CI khuyến khích merge nhỏ, thường xuyên và test liên tục.

Continuous Deployment (CD): Là bước tiếp theo, tự động deploy code đã pass test lên production. Với CD, mỗi commit vào main branch có thể tự động lên prod nếu pass hết pipeline. Điều này giúp giảm thời gian từ code đến tay người dùng xuống còn vài phút thay vì vài ngày.

2.4.2. GitHub Actions

GitHub Actions là nền tảng CI/CD tích hợp sẵn trong GitHub. Workflows được định nghĩa bằng các file YAML trong repo, mỗi workflow gồm các jobs và steps để build, test và deploy. Các thành phần chính gồm:

- Workflow: File YAML trong `.github/workflows/` định nghĩa toàn bộ pipeline.
- Runner: Máy chủ thực thi các jobs. GitHub cung cấp hosted runners (Ubuntu, Windows, macOS) miễn phí với giới hạn thời gian; có thể dùng self-hosted runner nếu cần cấu hình đặc biệt.
- Secrets: Cho phép lưu dữ liệu nhạy cảm như thông tin Docker dưới dạng encrypted secrets. Trong workflow, chỉ cần tham chiếu mà không lộ dữ liệu plaintext.

Tại sao dùng GitHub Actions: Đồ án chọn GitHub Actions vì tích hợp sẵn với repo, không cần setup CI server riêng như Jenkins. Mỗi lần push code lên branch dev hoặc main, workflow tự động trigger, build Docker image với tag là commit SHA, và push lên Docker Hub. Quá trình này hoàn toàn tự động, dev chỉ cần push code.

2.5. GitOps với Argo CD

GitOps là phương pháp triển khai ứng dụng, trong đó Git repo là single source of truth cho cả code và cấu hình hạ tầng. Thay vì chạy kubectl apply thủ công, các công cụ GitOps tự động đồng bộ trạng thái cluster với config trong Git.

Argo CD là công cụ GitOps cho Kubernetes, theo dõi Git repo chứa Kubernetes manifests (Deployment, Service, Ingress) và tự động áp dụng thay đổi lên cluster khi phát hiện sự khác biệt.

Cách hoạt động: Khi commit manifests vào Git, Argo CD pull config, so sánh với trạng thái cluster và sync nếu khác biệt. Nếu ai đó thay đổi trực tiếp trên cluster, Argo CD phát hiện drift và có thể tự revert về trạng thái đúng trong Git.

Lý do dùng Argo CD: Với hai cluster riêng cho dev và prod, Argo CD quản lý deployment declarative. Mỗi cluster có instance riêng theo dõi branch/folder tương ứng. Muốn update app version, chỉ cần sửa image tag trong manifest và commit; Argo CD tự động deploy đúng cluster. Rollback đơn giản: revert commit và Argo CD deploy lại version cũ. Cách này đảm bảo mọi thay đổi được track trong Git history và hai môi trường hoàn toàn độc lập.

2.6. NGINX Ingress Controller

Trong Kubernetes, Service chỉ cung cấp internal networking. Để expose ứng dụng ra Internet với domain name và HTTPS, cần Ingress Controller.

NGINX Ingress Controller là implementation phổ biến nhất của Kubernetes Ingress. Nó chạy như một pod trong cluster, nhận traffic từ external load balancer và route đến các Service dựa trên rules trong Ingress resources.

Khi deploy Ingress Controller, nó tạo một LoadBalancer Service với public IP. Bạn định nghĩa Ingress resource chứa các rules; NGINX Ingress Controller sẽ đọc config và tự động cấu hình Nginx để route traffic đúng.



Hình 2.2: Kubernetes Ingress Architecture

2.7. Giám sát hệ thống (Monitoring)

2.7.1. Prometheus

Prometheus là hệ thống monitoring và alerting mã nguồn mở, phù hợp môi trường Kubernetes. Nó pull metrics từ ứng dụng qua endpoint, lưu vào time-series database và tự động discover targets trong cluster. Hỗ trợ alerting theo rules, ví dụ CPU > 80% trong 5 phút sẽ gửi thông báo qua Alertmanager.

2.7.2. Grafana

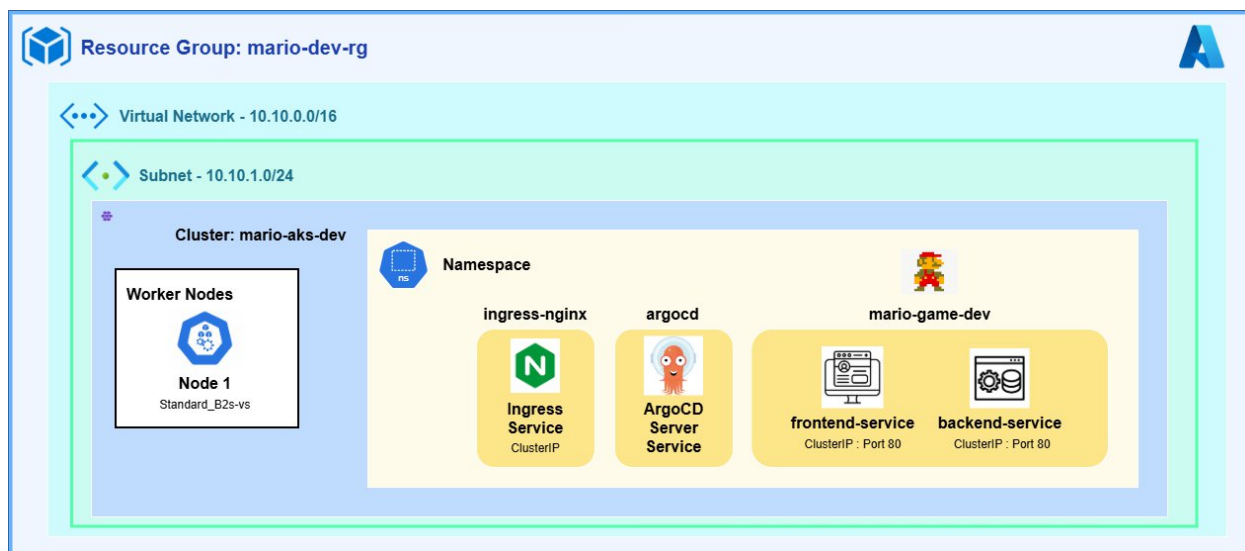
Grafana là công cụ visualize metrics, kết nối với Prometheus làm data source, cho phép tạo dashboards, panels với charts, graphs, tables hiển thị metrics theo thời gian thực. Có thể theo dõi cluster (tổng số pods, CPU/memory nodes) hoặc app-specific (request rate, error rate, response time của Mario app).

Lý do dùng Prometheus & Grafana: Đây là standard trong Kubernetes monitoring. Prometheus thu thập metrics, Grafana visualize để dễ theo dõi. Với đồ án, setup basic monitoring giúp track resource usage và health của ứng dụng, đảm bảo hệ thống chạy ổn định.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc hệ thống tổng quan

3.1.1. Kiến trúc môi trường Development



Hình 3.1: Kiến trúc môi trường Development

Thành phần	Cấu hình
Resource Group	mario-dev-rg
AKS Cluster	mario-aks-dev
Virtual Network	10.10.0.0/16
AKS Subnet	10.10.1.0/24
Worker Nodes	1 node
VM Size	Standard_B2s_v2

Bảng 3.1: Cấu hình môi trường Development

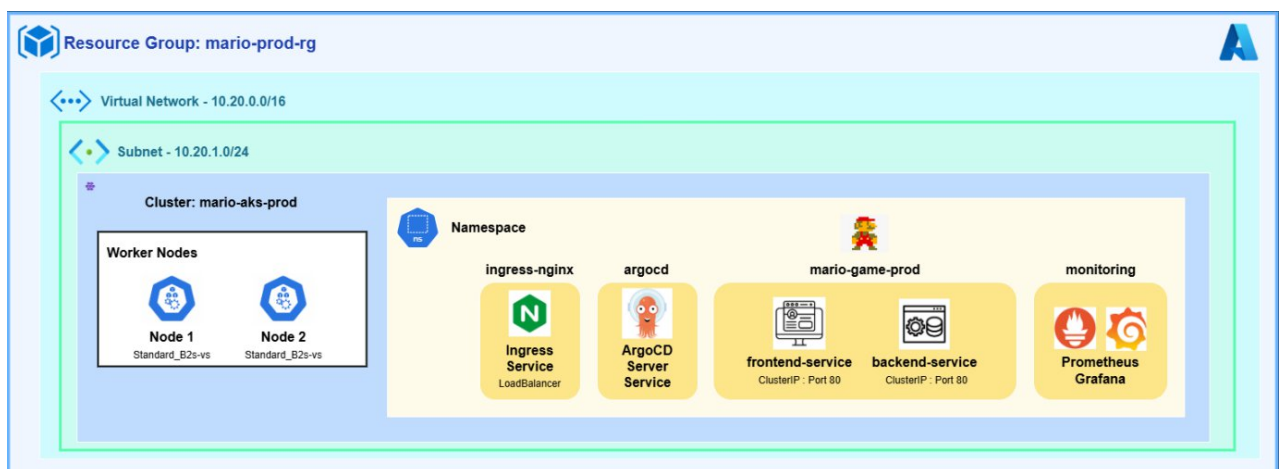
Môi trường Development triển khai tại Azure East Asia với một VNet, subnet cho AKS. Cluster dùng 1 worker node, Azure CNI và Standard Load Balancer để tối ưu chi phí.

Các thành phần trong cluster được phân tách thành các namespace riêng biệt để dễ quản lý và cô lập tài nguyên.

- Namespace ingress-nginx: Chứa NGINX Ingress Controller chịu trách nhiệm routing traffic vào các service trong cluster. Ingress Controller được expose thông qua Service type ClusterIP, không cấp public IP, chỉ cho phép truy cập nội bộ.
- Namespace argocd: Chứa ArgoCD Server theo dõi Git repository để tự động sync và deploy ứng dụng. ArgoCD được cấu hình auto-sync và self-heal, đảm bảo trạng thái cluster luôn khớp với cấu hình được định nghĩa trong Git.
- Namespace mario-game-dev: Chứa các workloads chính của ứng dụng Mario Game bao gồm Frontend service, Backend service và Ingress resource để định nghĩa routing rules.

Ingress Controller dùng Service type ClusterIP, chỉ truy cập được qua kubectl port-forward hoặc từ các pod trong cluster, phù hợp cho testing nội bộ, không expose ra Internet.

3.1.2. Kiến trúc môi trường Production



Hình 3.2: Kiến trúc môi trường Production

Môi trường Production được triển khai với Resource Group riêng biệt tại cùng Azure Region. Virtual Network sử dụng dải địa chỉ tách biệt hoàn toàn với môi trường Development. AKS Cluster bao gồm hai worker nodes để đảm bảo High Availability - khi một node gặp sự cố, các pods có thể được reschedule sang node còn lại, đảm bảo uptime cho production. Azure Load Balancer với public IP được tạo tự động khi deploy Ingress Controller, cho phép truy cập từ Internet vào ứng dụng.

Thành phần	Cấu hình
Resource Group	mario-prod-rg
AKS Cluster	mario-aks-prod
Virtual Network	10.20.0.0/16
AKS Subnet	10.20.1.0/24
Worker Nodes	2 nodes
VM Size	Standard_B2s_v2

Bảng 3.2: Cấu hình môi trường Production

Tổ chức Namespaces:

- Namespace ingress-nginx: NGINX Ingress Controller với Service type LoadBalancer được gắn với Azure Load Balancer public IP. External traffic policy được set là Local để preserve source IP của clients. Đây là entry point duy nhất để users truy cập ứng dụng từ Internet.
- Namespace argocd: ArgoCD Server theo dõi Git repository branch chính. Namespace này có thêm Ingress resource expose webhook endpoint để nhận thông báo từ GitHub, giúp ArgoCD tự động sync ngay khi có commit mới.
- Namespace mario-game-prod: Chứa các workloads chính với cấu hình cao hơn môi trường Development. Mỗi service được deploy với nhiều replicas để đảm

bảo High Availability. Kubernetes scheduler tự động phân bổ pods lên các nodes khác nhau để tránh single point of failure.

Luồng request trong Production: User → Internet → Azure Load Balancer (Public IP) → NGINX Ingress Controller → Service (Frontend/Backend) → Pods (distributed trên 2 nodes). Với thiết kế multi-node và multi-replica, hệ thống đảm bảo khả năng chịu lỗi và tính sẵn sàng cao.

3.1.3. So sánh hai môi trường

Khía cạnh	Development	Production
Mục đích	Testing và phát triển	Phục vụ người dùng thực
High Availability	Không (1 node)	Có (2 nodes)
External Access	Không (ClusterIP)	Có (LoadBalancer + Public IP)
Resource Allocation	Thấp (tối ưu chi phí)	Cao (đảm bảo hiệu năng)

Bảng 3.3: So sánh 2 môi trường

Sự khác biệt chính giữa hai môi trường nằm ở khả năng mở rộng, tính sẵn sàng cao và cách tiếp cận bảo mật. Môi trường Development tập trung vào tối ưu chi phí và thuận tiện cho việc phát triển, trong khi Production được thiết kế để đảm bảo độ tin cậy, hiệu năng và khả năng chịu lỗi cao.

3.2. Thiết kế quy trình DevOps

3.2.1. Chiến lược phân nhánh Git

Hệ thống áp dụng Git Flow nhằm đảm bảo tách biệt rõ ràng giữa môi trường phát triển và môi trường production, đồng thời kiểm soát tốt chất lượng mã nguồn trong suốt vòng đời phát triển.

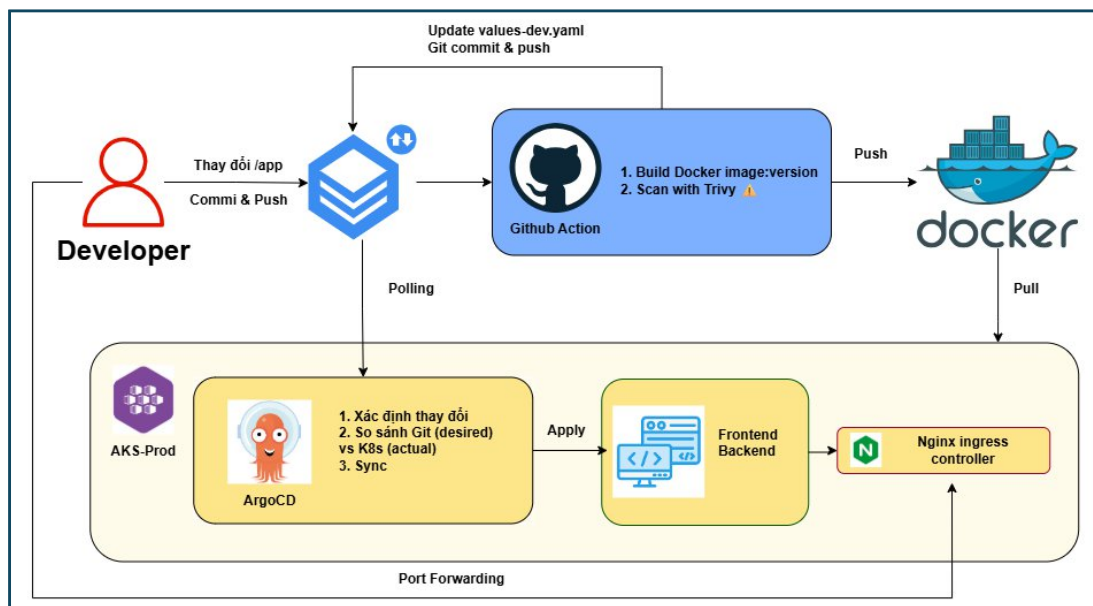
Hai nhánh chính được sử dụng là develop và main. Nhánh develop đóng vai trò là nhánh tích hợp, nơi tập trung toàn bộ thay đổi trong quá trình phát triển và được deploy tự động lên môi trường Development để kiểm thử liên tục. Nhánh main đại diện

cho mã nguồn ổn định ở mức production, chỉ chứa các phiên bản đã được kiểm thử và phê duyệt đầy đủ trước khi phát hành.

Trong quá trình phát triển, developer có thể tạo các nhánh tính năng (feature branches) từ nhánh develop để thực hiện các chức năng mới. Sau khi hoàn thành, các nhánh này được merge ngược trở lại develop và xóa bỏ nhằm giữ cho repository gọn gàng.

Quy trình làm việc được kiểm soát thông qua phân quyền rõ ràng: developer chỉ làm việc trên develop hoặc feature branches, trong khi việc merge vào main và phát hành phiên bản production chỉ được thực hiện bởi Admin hoặc Tech Lead thông qua Pull Request.

3.2.2. Quy trình CI/CD cho môi trường Development



Hình 3.3: Luồng hoạt động Development

Pipeline CI/CD cho môi trường Development được kích hoạt tự động khi developer push code lên nhánh develop. Quy trình này đảm bảo việc build, kiểm tra bảo mật và triển khai được thực hiện liên tục, giúp phát hiện lỗi sớm trong quá trình phát triển.

Sau khi code được push, hệ thống CI tiến hành build Docker image, thực hiện quét bảo mật để phát hiện các lỗ hổng nghiêm trọng, sau đó push image lên registry. Manifest GitOps tương ứng được cập nhật tự động để phản ánh phiên bản image mới.

ArgoCD theo dõi repository và tự động đồng bộ trạng thái mong muốn từ Git xuống AKS cluster Development, thực hiện rolling update các workload. Developer có thể xác minh kết quả triển khai thông qua ArgoCD hoặc truy cập trực tiếp vào ứng dụng để kiểm thử chức năng.

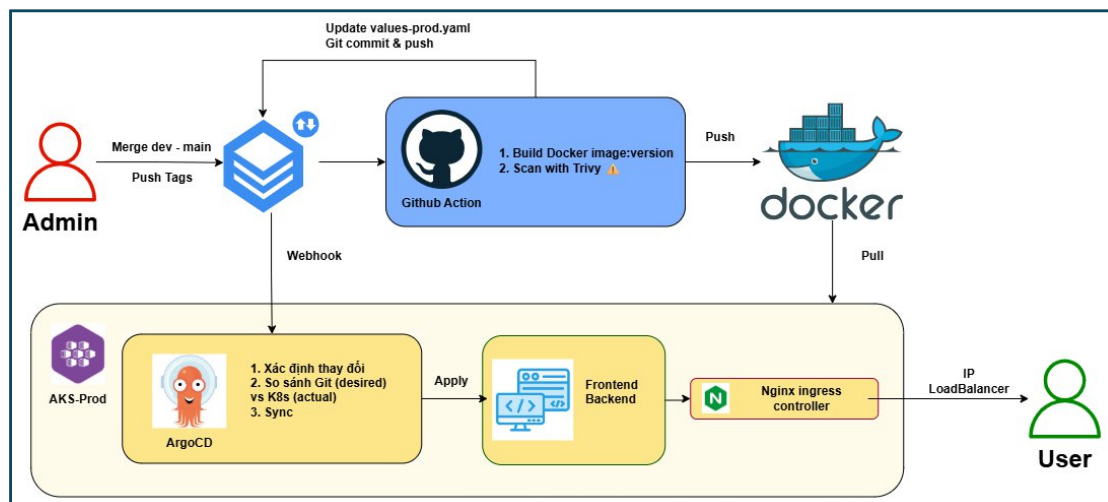
Toàn bộ pipeline được thiết kế để tối ưu tốc độ, với thời gian triển khai chỉ trong vài phút kể từ khi push code.

3.2.3. Quy trình CI/CD cho môi trường Production

Pipeline Production được thiết kế cẩn trọng và có kiểm soát, không tự động kích hoạt từ thay đổi mã nguồn. Việc triển khai production chỉ bắt đầu khi Admin hoặc Tech Lead chủ động tạo Git tag phát hành trên nhánh main.

Sau khi tag được tạo, pipeline CI tiến hành build image từ phiên bản đã được release, thực hiện quét bảo mật, gắn thẻ theo semantic version và push lên registry. Các manifest production được cập nhật tương ứng để phản ánh phiên bản mới.

ArgoCD nhận tín hiệu thay đổi và triển khai application lên AKS Production cluster theo chiến lược rolling update, đảm bảo hệ thống luôn sẵn sàng và không gây gián đoạn dịch vụ. Sau khi deploy, trạng thái hệ thống được giám sát để đảm bảo ứng dụng hoạt động ổn định.



Hình 3.4: Luồng hoạt động Production

3.2.4. GitOps với ArgoCD

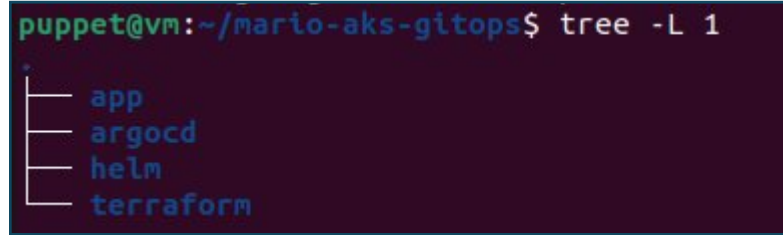
Hệ thống áp dụng GitOps với ArgoCD đóng vai trò là công cụ Continuous Deployment trung tâm. Mỗi service trên từng môi trường được định nghĩa như một application riêng biệt, với Git repository là single source of truth.

ArgoCD tự động đồng bộ trạng thái từ Git xuống cluster, đồng thời hỗ trợ các cơ chế tự phục hồi và dọn dẹp tài nguyên không còn tồn tại trong repository. Điều này giúp ngăn chặn configuration drift và đảm bảo tính nhất quán giữa môi trường thực tế và cấu hình khai báo.

Đối với môi trường Production, ArgoCD được cấu hình webhook để nhận thông báo trực tiếp từ GitHub, giúp giảm độ trễ triển khai so với cơ chế polling mặc định.

CHƯƠNG 4: HIỆN THỰC VÀ TRIỂN KHAI

4.1. Chuẩn bị môi trường & Source code

A terminal window with a dark purple background. The prompt is 'puppet@vm:~/mario-aks-gitops\$'. The command 'tree -L 1' has been executed, showing a directory tree with four subdirectories: 'app', 'argocd', 'helm', and 'terraform', each preceded by a vertical line and a horizontal line connecting to the root directory.

```
puppet@vm:~/mario-aks-gitops$ tree -L 1
.
├── app
├── argocd
├── helm
└── terraform
```

Hình 4.1: Cấu trúc thư mục của Repo

Dự án được tổ chức theo cấu trúc thư mục phân tách rõ ràng giữa mã nguồn ứng dụng, hạ tầng, cấu hình triển khai và GitOps. Mỗi thư mục đảm nhiệm một vai trò riêng trong quy trình DevOps, giúp dễ dàng quản lý, mở rộng và bảo trì hệ thống. Chi tiết:

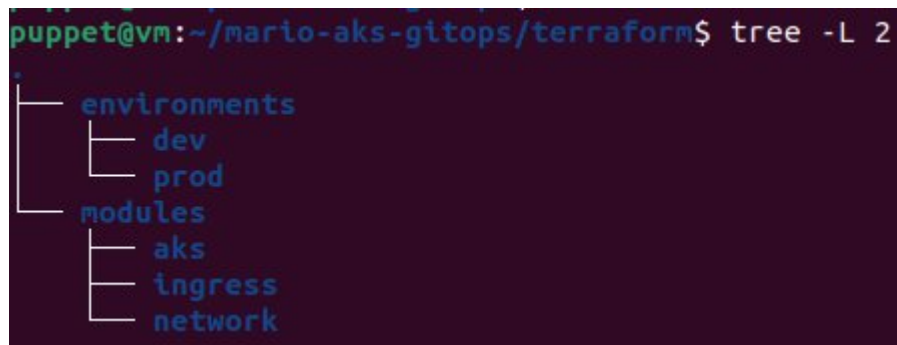
- `app/`: chứa mã nguồn ứng dụng frontend và backend, cùng các Dockerfile phục vụ quá trình containerization
- `terraform/`: chứa mã nguồn Infrastructure as Code dùng Terraform để triển khai hạ tầng trên cloud (network, AKS, ingress)
- `helm/`: chứa các Helm chart cho từng service và từng môi trường (dev, prod), phục vụ việc triển khai ứng dụng lên Kubernetes.
- `argocd/`: chứa các manifest ArgoCD Application định nghĩa cơ chế GitOps và tự động đồng bộ cấu hình từ Git xuống cluster Kubernetes.
- `.github/workflows/`: chứa các workflow CI/CD được cấu hình bằng GitHub Actions.

Toàn bộ quá trình triển khai và vận hành hệ thống được thực hiện từ máy ảo chạy Ubuntu Server 22.04 LTS, đóng vai trò là máy điều khiển (control machine). Trên máy này, các công cụ DevOps như Docker, Terraform, kubectl, Helm và ArgoCD CLI được cài đặt để phục vụ việc quản lý hạ tầng, đóng gói ứng dụng và tương tác với Kubernetes cluster. Cấu trúc từng thư mục và quy trình triển khai sẽ được trình bày lần lượt trong các mục tiếp theo của chương này; các bước cài đặt môi trường và cấu hình công cụ được mô tả trong repository mã nguồn của dự án.

4.2. Triển khai Hạ tầng với IaC (Terraform)

4.2.1. Cấu trúc thư mục

Terraform code được tổ chức theo mô hình module hóa, tách biệt giữa cấu hình môi trường và các modules tái sử dụng. Cấu trúc này giúp dễ dàng quản lý nhiều môi trường khác nhau mà không cần duplicate code.



Hình 4.2: Cấu trúc thư mục thư mục terraform

Thư mục terraform/ được tổ chức theo hướng module hóa và tách biệt môi trường, bao gồm hai phần chính:

- environments/: chứa cấu hình triển khai cho từng môi trường (dev, prod).
- modules/: chứa các module Terraform tái sử dụng cho từng thành phần hạ tầng.

Mỗi environment (dev/prod) có file main.tf gọi các modules và file terraform.tfvars chứa giá trị cụ thể cho môi trường đó. Các modules trong thư mục modules/ chứa logic chung, nhận parameters qua variables và tái sử dụng được cho cả dev lẫn prod.

4.2.2. Module AKS

Module này tạo Azure Kubernetes Service cluster với các cấu hình cần thiết:

```
resource "azurerm_kubernetes_cluster" "aks" {
  name                = var.cluster_name
  location            = var.location
  resource_group_name = var.resource_group_name
  dns_prefix          = var.dns_prefix
  kubernetes_version  = var.kubernetes_version

  default_node_pool {
    name         = "default"
    node_count   = var.node_count
    vm_size      = var.vm_size
    vnet_subnet_id = var.subnet_id
  }

  identity { type = "SystemAssigned" }

  network_profile {
    network_plugin = "azure"
    load_balancer_sku = "standard"
  }

  tags = var.tags
}
```

Hình 4.3: Code file main.tf của module AKS

Trong đó default_node_pool định nghĩa số lượng nodes và VM size. Identity dùng SystemAssigned để AKS tự động tạo managed identity để truy cập các Azure resources khác. Network profile dùng Azure CNI plugin để pods nhận IP trực tiếp từ VNet, giúp network performance tốt hơn.

4.2.3. Module Network

Module network tạo Virtual Network và Subnet riêng cho AKS:

```
resource "azurerm_virtual_network" "vnet" {
  name                = "${var.cluster_name}-vnet"
  location            = var.location
  resource_group_name = var.resource_group_name
  address_space       = var.vnet_address_space
  tags               = var.tags
}

resource "azurerm_subnet" "aks" {
  name                = var.subnet_name
  resource_group_name = var.resource_group_name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes    = var.aks_subnet_prefix
}
```

Hình 4.4: Code file main.tf của module network

VNet cung cấp private network cho cluster. Môi trường dev dùng range 10.10.0.0/16, trong khi prod dùng 10.20.0.0/16 để đảm bảo hai môi trường hoàn toàn tách biệt về mặt network.

4.2.4. Module Ingress

Module này deploy NGINX Ingress Controller bằng Helm chart:

```
resource "helm_release" "nginx_ingress" {
  name           = "nginx-ingress"
  repository     = "https://kubernetes.github.io/ingress-nginx"
  chart         = "ingress-nginx"
  namespace     = "ingress-nginx"
  create_namespace = true
  version       = "4.9.0"

  set {
    name = "controller.service.type"
    value = var.service_type
  }

  dynamic "set" {
    for_each = var.service_type != "ClusterIP" ? [1] : []
    content {
      name = "controller.service.externalTrafficPolicy"
      value = var.external_traffic_policy
    }
  }

  depends_on = [
    var.aks_cluster_id
  ]
}
```

Hình 4.5: Code file main.tf của module ingress

Block dynamic "set" chỉ apply khi service_type không phải ClusterIP. Điều này cho phép môi trường dev dùng ClusterIP (không expose ra ngoài) trong khi prod dùng LoadBalancer với external IP. depends_on đảm bảo AKS cluster đã sẵn sàng trước khi cài Helm chart.

4.2.5. Cấu hình môi trường Development

File environments/dev/terraform.tfvars định nghĩa các giá trị cụ thể cho môi trường dev:

```
resource_group_name = "mario-dev-rg"
location            = "East Asia"
cluster_name        = "mario-aks-dev"
dns_prefix           = "mario-dev"
#kubernetes_version = "1.28"
node_count           = 1
vm_size             = "Standard_B2s_v2"

vnet_address_space = ["10.10.0.0/16"]
aks_subnet_prefix  = ["10.10.1.0/24"]

tags = {
  Environment = "Development"
  Project     = "Mario-Game"
  ManagedBy   = "Terraform"
}
```

Hình 4.6: Code file .tfvars của môi trường development

Môi trường dev chỉ dùng 1 node với VM size nhỏ (Standard_B2s_v2) để tiết kiệm chi phí. Ingress được cấu hình là ClusterIP, nghĩa là không tạo external IP, phù hợp cho testing nội bộ.

4.2.6. Cấu hình môi trường Production

File environments/prod/terraform.tfvars cho production:

```
resource_group_name = "mario-prod-rg"
location            = "East Asia"
cluster_name        = "mario-aks-prod"
dns_prefix           = "mario-prod"
#kubernetes_version = "1.28"
node_count           = 2
vm_size             = "Standard_B2s_v2"

vnet_address_space = ["10.20.0.0/16"]
aks_subnet_prefix  = ["10.20.1.0/24"]

tags = {
  Environment = "Production"
  Project     = "Mario-Game"
  ManagedBy   = "Terraform"
}
```

Hình 4.7: Code file .tfvars của môi trường production

Khác với dev, prod dùng 2 nodes để đảm bảo high availability. Nếu một node fail, ứng dụng vẫn chạy trên node còn lại. Ingress type là LoadBalancer với

external_traffic_policy set thành Local, tạo public IP để user có thể truy cập game từ Internet.

4.2.7. Orchestration với main.tf của môi trường

File main.tf trong mỗi môi trường đóng vai trò orchestrating, kết nối các modules lại với nhau theo đúng thứ tự phụ thuộc. File này bắt đầu bằng việc khai báo providers và tạo Resource Group, sau đó gọi từng module theo thứ tự:

```
resource "azurerm_resource_group" "rg" {
  name     = var.resource_group_name
  location = var.location
  tags     = var.tags
}

module "network" {
  source = "../../modules/network"

  resource_group_name = azurerm_resource_group.rg.name
  location             = azurerm_resource_group.rg.location
  cluster_name         = var.cluster_name
  subnet_name          = "mario-prod-subnet"
  vnet_address_space   = var.vnet_address_space
  aks_subnet_prefix    = var.aks_subnet_prefix
  tags                 = var.tags
}

module "aks" {
  source = "../../modules/aks"

  resource_group_name = azurerm_resource_group.rg.name
  location             = azurerm_resource_group.rg.location
  cluster_name         = var.cluster_name
  dns_prefix           = var.dns_prefix
  kubernetes_version   = var.kubernetes_version
  node_count           = var.node_count
  vm_size              = var.vm_size
  subnet_id            = module.network.aks_subnet_id
  tags                 = var.tags
}
```

Hình 4.8: Code file main.tf của môi trường development (1)

Module network được gọi trước để tạo VNet và Subnet. Output aks_subnet_id của module này được truyền vào module aks thông qua parameter subnet_id, đảm bảo cluster được deploy vào đúng subnet vừa tạo. Module aks sau đó output thông tin kube config để configure Helm provider:

```
# Helm Provider
provider "helm" {
  kubernetes {
    host      = module.aks.kube_config_host
    client_certificate = base64decode(module.aks.kube_config_client_certificate)
    client_key   = base64decode(module.aks.kube_config_client_key)
    cluster_ca_certificate = base64decode(module.aks.kube_config_cluster_ca_certificate)
  }
}

# Ingress Module
module "ingress" {
  source      = "../../modules/ingress"
  aks_cluster_id = module.aks.cluster_id
  service_type = "ClusterIP"
}
```

Hình 4.9: Code file main.tf của môi trường development (2)

Helm provider sử dụng credentials từ AKS cluster để deploy Ingress Controller. Sự khác biệt giữa dev và prod thể hiện qua giá trị service_type: dev dùng "ClusterIP" trong khi main.tf của prod có thêm external_traffic_policy = "Local" khi gọi module ingress.

4.2.8. Quy trình triển khai

Để deploy infrastructure lên Azure, chạy các lệnh Terraform theo thứ tự:

```
cd terraform/environments/dev
terraform init
terraform plan
terraform apply
```

Hình 4.10: Các lệnh để deploy infrastructure môi trường development

Lệnh terraform init download Azure provider và Helm provider cần thiết. terraform plan so sánh config với state hiện tại, cho biết sẽ tạo bao nhiêu resources mà không thực sự thay đổi gì. Sau khi review plan output, chạy terraform apply để tạo thật sự AKS cluster, VNet, Subnet và deploy NGINX Ingress. Toàn bộ quá trình này tự động, không cần click trên Azure Portal.

Nếu cần thay đổi cấu hình sau này, chỉ cần sửa file .tfvars hoặc module code, chạy lại terraform plan để xem sẽ thay đổi gì, rồi terraform apply. Terraform tự động so sánh trạng thái hiện tại với config mới và chỉ update những gì cần thiết, đảm bảo infrastructure luôn khớp với code.

4.3. Đóng gói ứng dụng (Containerization)

4.3.1. Tổng quan về ứng dụng

Ứng dụng Mario Game được xây dựng theo kiến trúc microservices với hai thành phần chính:

- Frontend: Ứng dụng HTML5 Mario chạy trên trình duyệt; backend chỉ được bổ sung để mô phỏng kiến trúc DevOps. Frontend gọi /api/hello qua NGINX Ingress (path-based routing).

```
document.getElementById("call-backend").addEventListener("click", async () => {
  const resultEl = document.getElementById("api-result");
  resultEl.textContent = "Loading...";

  try {
    const response = await fetch("/api/hello");
    const data = await response.json();
    resultEl.textContent = data.message;
  } catch (err) {
    resultEl.textContent = "Error calling API";
  }
});
```

Hình 4.11: Frontend thực hiện gọi API tới Backend

- Backend: REST API FastAPI cung cấp /api/hello trả về version, không tham gia logic game, dùng minh họa mô hình frontend-backend.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/api/hello")
def hello():
    return {"message": "Hello from API v3.0.0"}
```

Hình 4.12: File backend app.py

Việc tách frontend và backend thành hai service riêng biệt giúp hệ thống dễ mở rộng, bảo trì và triển khai độc lập từng thành phần.

4.3.2. Dockerfile cho Frontend

Frontend sử dụng NGINX Alpine image làm base, image này nhẹ (chỉ ~5MB) và đủ để serve static files:

```
FROM nginx:alpine

COPY . /usr/share/nginx/html/

EXPOSE 80

HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD wget --quiet --tries=1 --spider http://localhost/ || exit 1

CMD ["nginx", "-g", "daemon off;"]
```

Hình 4.13: Dockerfile cho Frontend

Dockerfile copy toàn bộ frontend (HTML, CSS, JS) vào thư mục mặc định của NGINX. HEALTHCHECK dùng wget --spider để kiểm tra HTTP status; nếu thất bại liên tiếp, Kubernetes sẽ tự động restart container.

4.3.3. Dockerfile cho Backend

Backend cần Python runtime và dependencies từ requirements.txt:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

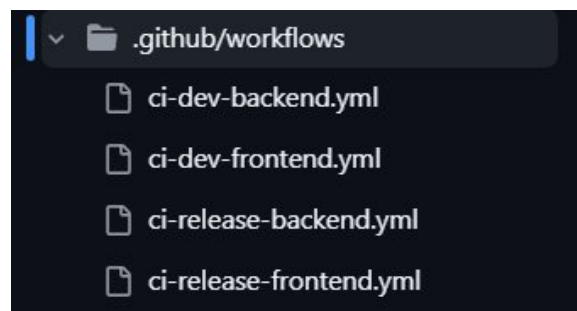
Hình 4.14: Dockerfile cho Backend

Dockerfile sử dụng Python 3.11-slim làm base image và sắp xếp lệnh để tận dụng Docker layer caching: cài dependencies trước, copy source sau, giúp tăng tốc quá trình build khi code thay đổi.

4.4. Xây dựng Pipeline CI/CD

Pipeline CI/CD là trung tâm của quy trình DevOps, tự động hóa toàn bộ quá trình từ code commit đến deployment trên production. Hệ thống sử dụng GitHub Actions cho giai đoạn CI (Continuous Integration) và Argo CD cho giai đoạn CD (Continuous Deployment) theo mô hình GitOps.

4.4.1. Giai đoạn CI - Continuous Integration với GitHub Actions



Hình 4.15: Danh sách các CI workflows

GitHub Actions được sử dụng để tự động hóa các bước CI cho cả frontend và backend. Mặc dù có 4 workflows khác nhau nhưng các bước xử lý là tương đồng, chỉ khác về trigger, cách đặt tag và target environment. Một workflow CI tiêu chuẩn trong hệ thống bao gồm các bước sau:

- Checkout source code từ Git repository.
- Sinh image tag (commit SHA hoặc semantic version).
- Build Docker image bằng Docker Buildx.
- Scan bảo mật image bằng Trivy.
- Push image lên Docker Hub.
- Cập nhật image tag trong Helm values.
- Commit và push thay đổi cấu hình về Git.

Sau đây là ví dụ về CI - Dev Frontend Build:

```
name: CI - Dev Frontend Build
on:
  push:
    branches:
      - develop
    paths:
      - 'app/frontend/**'
```

Hình 4.16: Điều kiện trigger workflows CI - Dev Frontend

Đoạn cấu hình trên mô tả điều kiện trigger của workflow. Workflow này được kích hoạt khi có thao tác push lên nhánh develop và đồng thời có thay đổi trong thư mục app/frontend/.

```
- name: Login Docker Hub
  uses: docker/login-action@v3
  with:
    username: ${ secrets.DOCKERHUB_USERNAME }
    password: ${ secrets.DOCKERHUB_TOKEN }

- name: Build Docker image
  uses: docker/build-push-action@v5
  with:
    context: ./app/frontend
    load: true
    tags: |
      ${ env.IMAGE_NAME }:${ steps.vars.outputs.tag }
```

Hình 4.17: Đăng nhập vào Docker Hub và build Image

Bước trên thực hiện xác thực Docker Hub bằng thông tin được lưu trong GitHub Secrets (không expose trong code), sau đó build Docker image từ Dockerfile trong thư mục app/frontend. Option load: true giữ image trên local runner thay vì push ngay (để scan trước khi push).

```
- name: Scan image with Trivy
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: ${ env.IMAGE_NAME }:${ steps.vars.outputs.tag }
    format: 'table'
    exit-code: '1'
    ignore-unfixed: true
    vuln-type: 'os,library'
    severity: 'CRITICAL,HIGH'
```

Hình 4.18: Scan Image đã build với Trivy

Trivy scan image để tìm vulnerabilities trong OS packages và libraries. Với exit-code: '1', nếu phát hiện lỗi hỏng CRITICAL hoặc HIGH, workflow sẽ fail và không push image lên registry. Đây là security gate quan trọng để đảm bảo không deploy vulnerable images.

Sau khi image vượt qua bước scan, workflow tiến hành push image lên Docker Hub. Việc cập nhật file values-dev.yaml và commit thay đổi chỉ được thực hiện khi workflow ở trạng thái success, nhằm đảm bảo chỉ những image hợp lệ mới được deploy lên môi trường development.

```
- name: Push image to Docker Hub
  run: |
    docker push ${ env.IMAGE_NAME }}:${ steps.vars.outputs.tag }}

- name: Update values-dev.yaml
  if: success()
  run: |
    sed -i "s|tag: .*|tag: ${ steps.vars.outputs.tag }}|g" helm/mario-frontend/values-dev.yaml

- name: Commit changes
  if: success()
  run: |
    git config user.name "GitHub Actions Bot"
    git config user.email "actions@github.com"
    git add helm/mario-frontend/values-dev.yaml
    git commit -m "dev-frontend: update image tag ${ steps.vars.outputs.tag }}"
    git pull --rebase origin develop
    git push origin develop
```

Hình 4.19: Push Image và cập nhật helm value tương ứng

So sánh Dev vs Release workflows:

Khía cạnh	Dev Workflow	Release Workflow
Trigger	Push to develop branch	Push Git tag frontend/backend-v*.*.*
Image Tag	dev-<commit-sha> (dev-8f64ff4)	Semantic version (v2.0.0)
Target Branch	develop	main
Target Environment	Development	Production
Values File	values-dev.yaml	values-prod.yaml
Frequency	Mỗi commit vào develop	Manual trigger khi release

4.4.2. Giai đoạn CD - Continuous Deployment với Argo CD



Hình 4.20: Danh sách các Manifest ArgoCD Applications

Argo CD thực hiện Continuous Deployment theo mô hình GitOps. Thay vì pipeline CI push trực tiếp manifest lên cluster, Argo CD pull cấu hình từ Git và tự động đồng bộ trạng thái cluster với repository. Tổng quan cơ chế CD:

- Mỗi service (frontend, backend) và mỗi môi trường (dev, prod) là một Argo CD Application riêng (tổng cộng 4 Applications).
- Argo CD theo dõi nhánh develop cho Dev, nhánh main cho Production.
- Khi Helm values hoặc chart thay đổi, Argo CD tự động sync xuống cluster.
- Không cần thao tác kubectl apply thủ công (cần apply lần đầu).

Ví dụ Manifest Argo CD Application cho frontend ở môi trường development:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: mario-frontend-dev
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/puppet1507/mario-aks-gitops.git
    targetRevision: develop
    path: helm/mario-frontend
    helm:
      valueFiles:
        - values-dev.yaml
  destination:
    server: https://kubernetes.default.svc
    namespace: mario-game-dev
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
      allowEmpty: false
    syncOptions:
      - CreateNamespace=true
  retry:
    limit: 5
    backoff:
      duration: 5s
      factor: 2
      maxDuration: 3m
```

Hình 4.21: Manifest Argo CD Application cho frontend ở môi trường development

Application này theo dõi Helm chart mario-frontend trong Git repository tại branch develop, sử dụng file values-dev.yaml và triển khai ứng dụng vào namespace mario-game-dev trên cluster hiện tại. Đồng bộ được cấu hình ở chế độ tự động với các chính sách sync và retry đã khai báo.

Lưu ý: Mỗi application cần cấu hình đúng valueFiles tương ứng với môi trường và nhánh Git để Argo CD sync chính xác.

Với môi trường production, nếu cần trigger tự động từ GitHub, cần apply thêm Ingress để expose webhook endpoint ra public IP. Khi có push lên branch main, GitHub gửi webhook đến /api/webhook, Argo CD sẽ sync ngay lập tức thay vì chờ polling interval.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: argocd-ingress
  namespace: argocd
  annotations:
    nginx.ingress.kubernetes.io/ssl-passthrough: "true"
    nginx.ingress.kubernetes.io/backend-protocol: "HTTPS"
spec:
  ingressClassName: nginx
  rules:
  - http:
      paths:
      - path: /api/webhook
        pathType: Prefix
        backend:
          service:
            name: argocd-server
            port:
              number: 443
```

Hình 4.22: Nội dung *argocd-ingress-prod.yaml*

4.4.3. Helm Charts Structure

```
puppet@vm:~/mario-aks-gitops/helm$ tree -L 3
.
├── mario-backend
│   ├── Chart.yaml
│   ├── templates
│   │   ├── deployment.yaml
│   │   └── service.yaml
│   ├── values-dev.yaml
│   └── values-prod.yaml
└── mario-frontend
    ├── Chart.yaml
    ├── templates
    │   ├── deployment.yaml
    │   ├── ingress.yaml
    │   └── service.yaml
    ├── values-dev.yaml
    └── values-prod.yaml
```

Hình 4.23: Cấu trúc thư mục helm

Hệ thống sử dụng Helm Charts để quản lý và triển khai ứng dụng trên Kubernetes. Mỗi ứng dụng (frontend và backend) có một chart riêng với cấu trúc như sau:

- Chart.yaml: khai báo thông tin chart, tên, phiên bản, description.
- templates/: chứa các manifest Kubernetes, sử dụng giá trị từ values-*.yaml để tùy biến.

Báo cáo Đồ án Công nghệ DevOps và Ứng dụng

- values-dev.yaml và values-prod.yaml: lưu các giá trị cấu hình khác nhau cho môi trường Development và Production, bao gồm số replica, image tag, resource limits, health checks, service, ingress.

Trong Helm chart, sử dụng ba template chính:

- Deployment: định nghĩa Pod, container, số lượng replica, image và cấu hình health checks (liveness & readiness probe).

```
replicaCount: 1

image:
  repository: docker.io/puppet666/mario-frontend
  tag: dev-8f64ff4
  pullPolicy: IfNotPresent
```

Hình 4.24: Value phần deployment trong file value-dev frontend (1)

```
resources:
  requests:
    memory: "64Mi"
    cpu: "50m"
  limits:
    memory: "128Mi"
    cpu: "100m"

livenessProbe:
  path: /
  port: 80
  initialDelaySeconds: 30
  periodSeconds: 10

readinessProbe:
  path: /
  port: 80
  initialDelaySeconds: 5
  periodSeconds: 5
```

Hình 4.25: Value phần deployment trong file value-dev frontend (2)

- Service: tạo Service trong Kubernetes để expose các pod và load balance traffic, xác định port và selector.

```
service:
  name: mario-frontend-service
  type: ClusterIP
  port: 80
```

Hình 4.26: Value phần service trong file value-dev frontend

- Ingress (chỉ frontend): định nghĩa các rule routing từ bên ngoài vào cluster, giúp truy cập ứng dụng từ internet.

```
ingress:
  enabled: true
  annotations: {}
  backend:
    serviceName: mario-backend-service
    servicePort: 8000
```

Hình 4.26: Value phần ingress trong file value-dev frontend

Cấu hình triển khai cụ thể ở hai bảng dưới đây:

Component	Replicas	Image Repo	Image Tag	CPU Request	Memory Request	Service Port	Service Type	Ingress Enabled
Frontend	2	mario-frontend	v*.*.*	100m	128Mi	80	ClusterIP	True
Backend	2	mario-backend	v*.*.*	50m	64Mi	8000	ClusterIP	False

Bảng 4.2: So sánh value của backend và frontend môi trường production

Component	Replicas	Image Repo	Image Tag	CPU Request	Memory Request	Service Port	Service Type	Ingress Enabled
Frontend	1	mario-frontend	dev-sha	50m	64Mi	80	ClusterIP	True
Backend	1	mario-backend	dev-sha	25m	32Mi	8000	ClusterIP	False

Bảng 4.3: So sánh value của backend và frontend môi trường development

4.5. Triển khai Giám sát (Monitoring)

Monitoring được triển khai tùy nhu cầu môi trường, sử dụng Prometheus + Grafana qua Helm chart kube-prometheus-stack. Hệ thống giám sát giúp thu thập metrics của Kubernetes cluster, pods, services và hiển thị trên dashboard Grafana.

1. Thêm Helm repo và cập nhật

```
helm repo add prometheus-community https://prometheus-  
community.github.io/helm-charts  
helm repo update
```

2. Tạo namespace cho monitoring

```
kubectl create namespace monitoring
```

3. Cài kube-prometheus-stack

```
helm install prometheus prometheus-community/kube-prometheus-stack \\\n  --namespace monitoring \\\n  --set prometheus.prometheusSpec.serviceMonitorSelectorNilUsesHelmValues=false \\\n  --set grafana.adminPassword=admin123
```

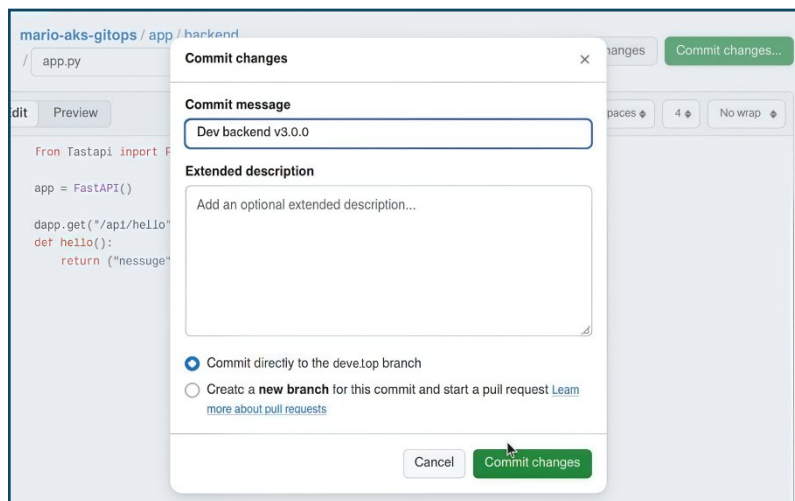
Hình 4.27: Cài đặt Prometheus và Grafana

CHƯƠNG 5. KẾT QUẢ VÀ ĐÁNH GIÁ

5.1. Kiểm thử luồng CI/CD tự động.

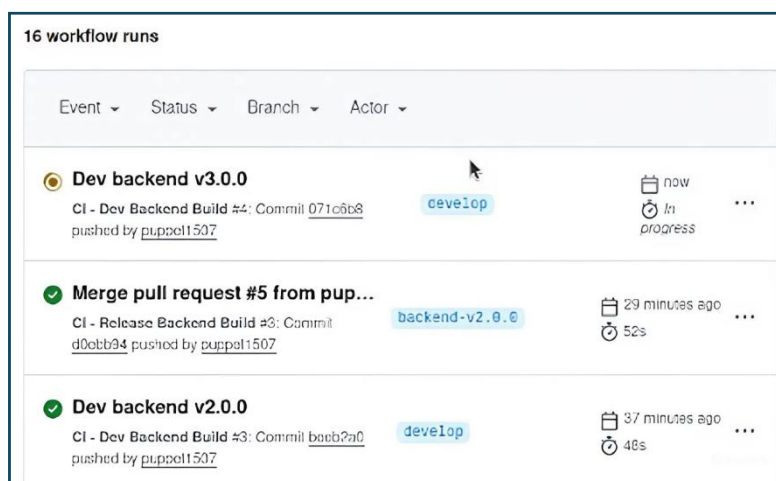
Quá trình kiểm thử được thực hiện bằng cách thay đổi code trong môi trường phát triển, sau đó push lên nhánh develop. Hệ thống CI tự động build, scan và push Docker image lên registry, sau đó Argo CD tự động triển khai lên cluster Dev. Khi pull request được merge hoặc Git tag được push, CI Release sẽ build, scan và push image lên registry, và Argo CD tự động triển khai phiên bản mới lên môi trường production.

- Bước 1: Chỉnh sửa code trong backend ở nhánh develop, sau đó commit và push.



Hình 5.1: Commit và push cập nhật backend ở nhánh develop

- Bước 2: Workflows của backend nhánh develop sẽ tự động chạy.



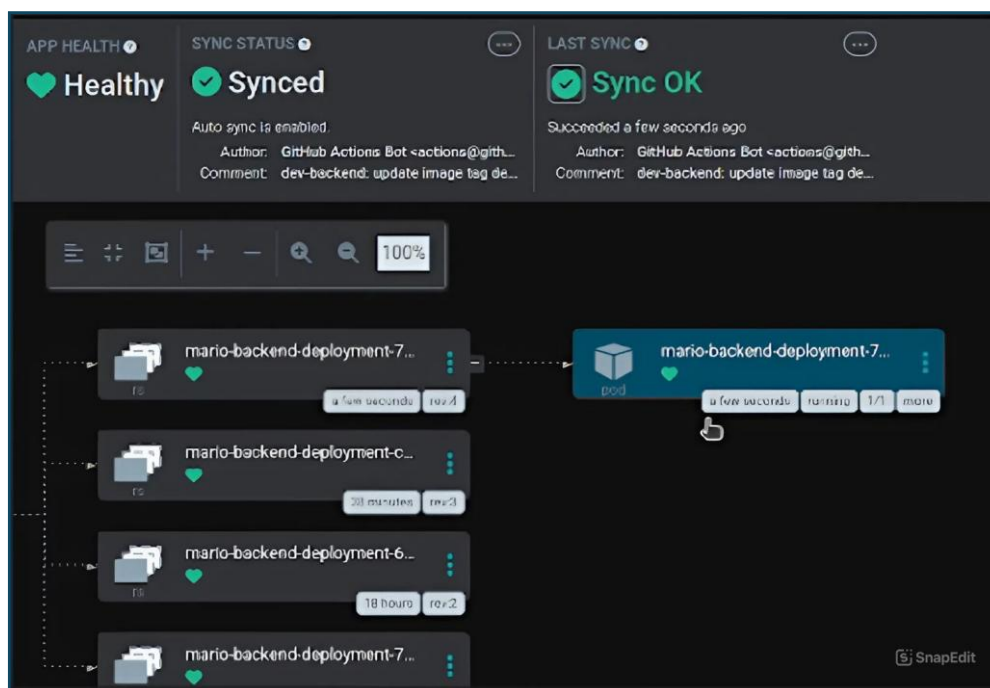
Hình 5.2: CI - dev Backend được trigger

- Bước 3: Kiểm tra trên Docker Hub sẽ có Image tag dev mới.

TAG			
 dev-071c6b8			
Last pushed less than a minute by puppet666			
<code>docker pull puppet666/mario-backend:dev-071c6b8</code>			
Digest	OS/ARCH	Last pull	Compressed size ①
84923b60ea21	linux/amd64	less than 1 day	69.66 MB

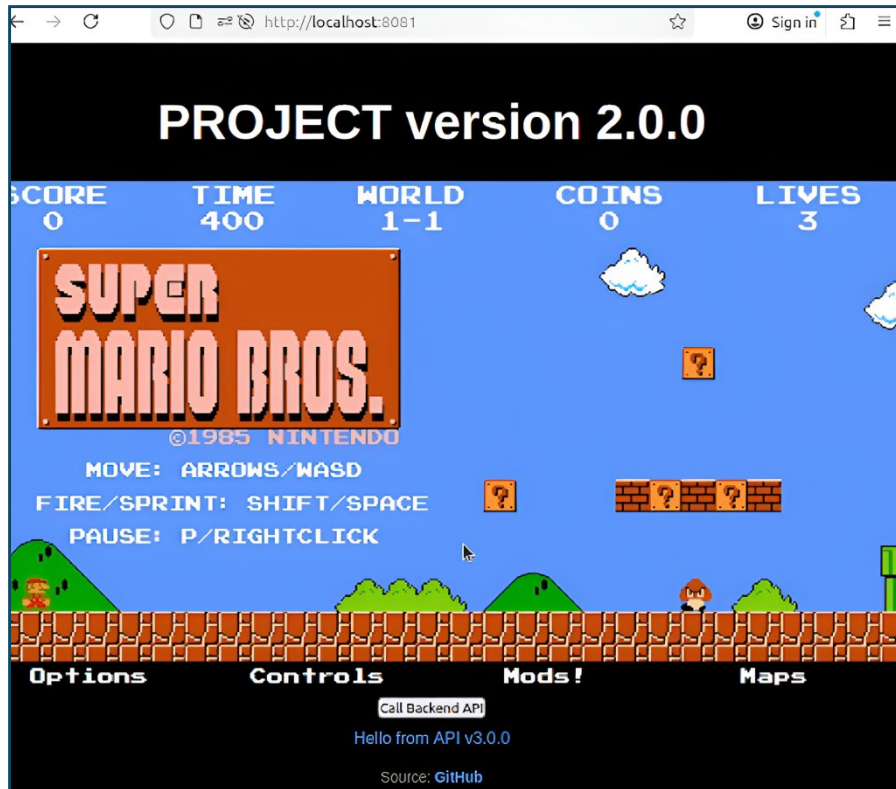
Hình 5.3: Trên Docker Hub có tag dev mới

- Bước 4: Chờ cơ chế Polling hoặc chủ động Sync ở ArgoCD



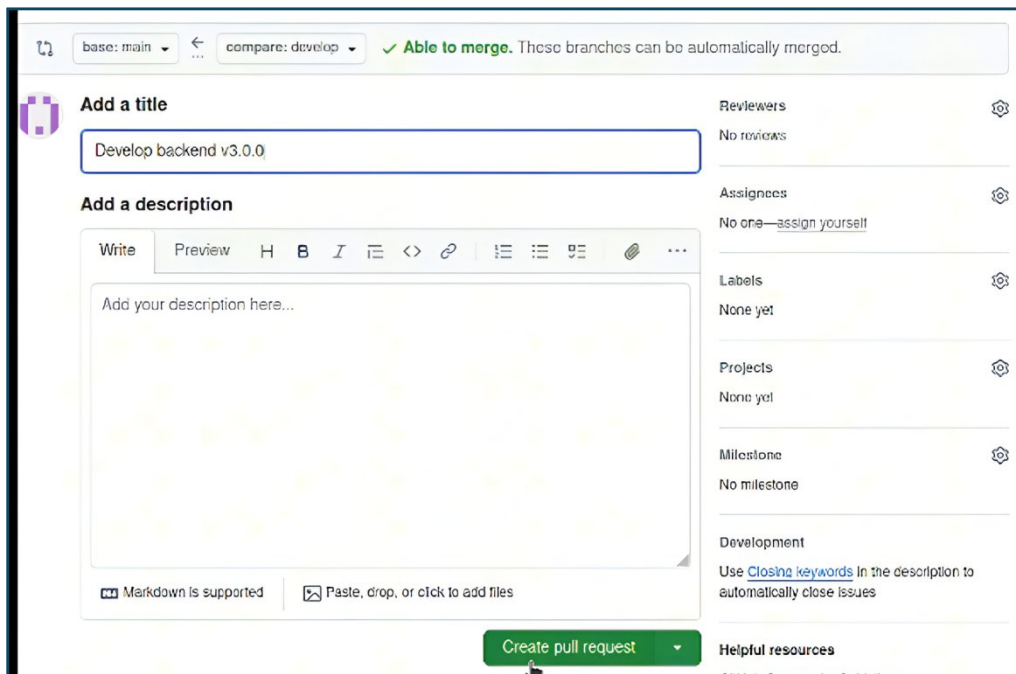
Hình 5.4: ArgoCD refresh và sync thành công

- Bước 5: Deploy backend development mới thành công.



Hình 5.5: Backend môi trường development đã được cập nhật

- Bước 6: Admin sẽ tạo Pull Request và Merge vào nhánh main



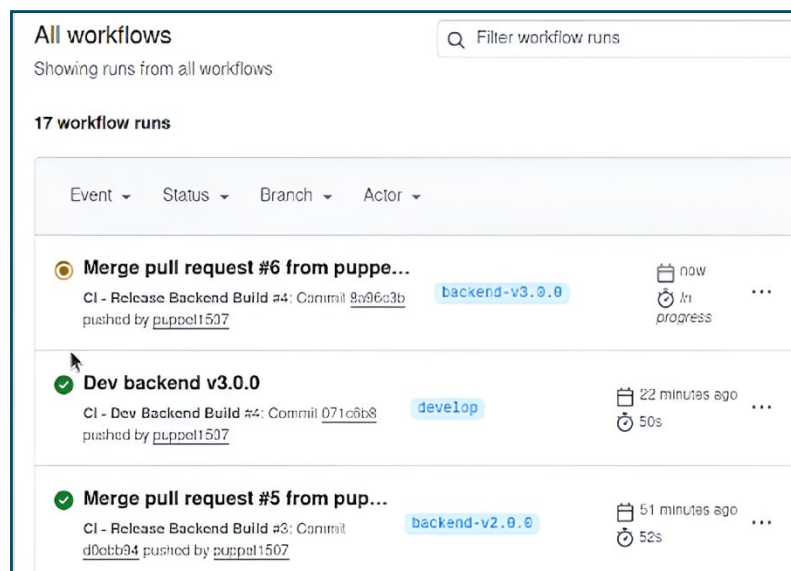
Hình 5.6: Tạo pull request và merge cập nhật từ develop vào main

- Bước 7: Cập nhật local code và push tag mới.

```
puppet@vm:~/mario-aks-gitops$ git pull origin main
From https://github.com/puppet1507/mario-aks-gitops
* branch      main      -> FETCH_HEAD
Updating d0ebb94..8a96c3b
Fast-forward
 app/backend/app.py          | 2 +-
 helm/mario-backend/Chart.yaml | 4 ++--
 helm/mario-backend/values-dev.yaml | 2 +-
 helm/mario-backend/values-prod.yaml | 2 +-
 4 files changed, 5 insertions(+), 5 deletions(-)
puppet@vm:~/mario-aks-gitops$ git tag backend-v3.0.0
puppet@vm:~/mario-aks-gitops$ git push origin backend-v3.0.0
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/puppet1507/mario-aks-gitops.git
* [new tag]          backend-v3.0.0 -> backend-v3.0.0
```

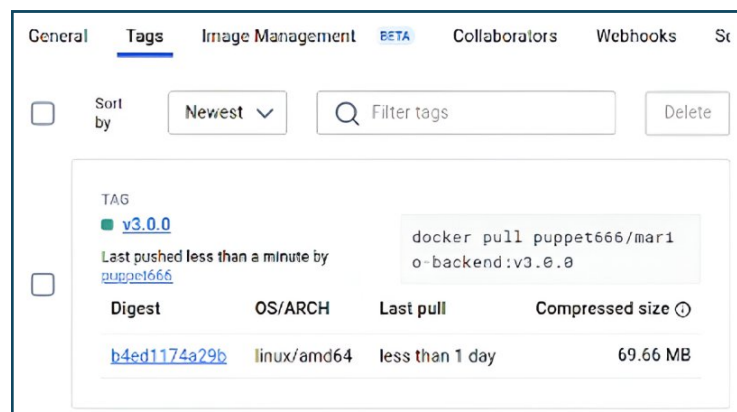
Hình 5.7: Tạo tag và push

- Bước 8: CI release backend sẽ tự động chạy.



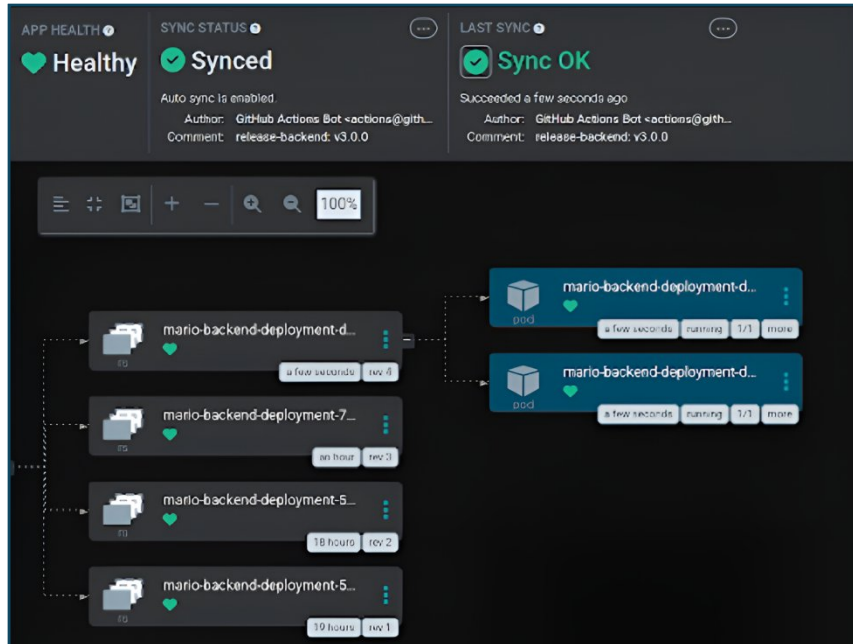
Hình 5.8: CI - Release Backend được trigger

- Bước 9: Trên Docker Hub có backend phiên bản mới.



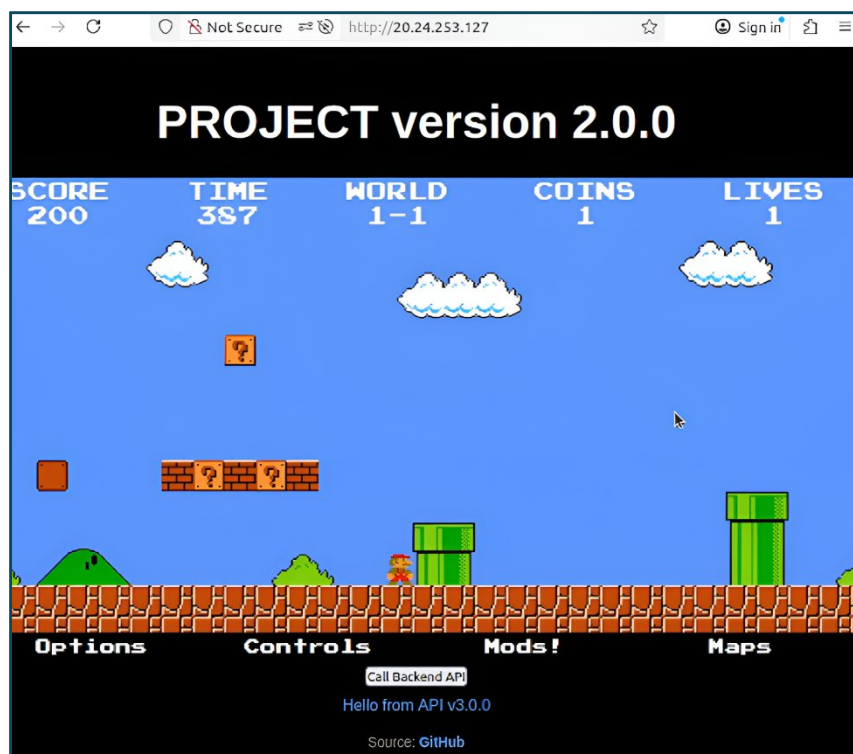
Hình 5.9: Trên Docker Hub có Tag version

- Bước 10: ArgoCD tự động Refresh bởi webhook và tự động Sync khi helm value được cập nhật.



Hình 5.10: ArgoCD refresh và Sync thành công

- Bước 11: Production đã cập nhật Backend phiên bản mới



Hình 5.11: Deploy Backend Production mới thành công

5.2. Đánh giá tổng quan

Quá trình CI/CD hoạt động ổn định và chính xác, đảm bảo rằng mọi thay đổi code ở môi trường phát triển được tự động build, kiểm tra bảo mật và deploy lên cluster Dev mà không cần can thiệp thủ công. Cơ chế release cho môi trường production hoạt động theo luồng GitOps, với việc pull request, merge và push tag kích hoạt tự động build, scan và deploy phiên bản mới mà vẫn giữ được tính nhất quán giữa Git repository và cluster.

Việc deploy trên cả Dev và Prod diễn ra mượt mà, các image được tag đúng phiên bản và cập nhật chính xác trong Helm values, giúp Argo CD đồng bộ cluster theo đúng cấu hình. Nhờ cơ chế self-heal và automated sync, hệ thống hạn chế tối đa lỗi do thay đổi thủ công, đồng thời cơ chế rollback qua GitOps cho phép quay về phiên bản trước nhanh chóng nếu xảy ra sự cố. Tổng thể, luồng CI/CD đã chứng minh khả năng tự động hóa toàn bộ pipeline từ code commit đến triển khai production, giảm thiểu rủi ro và tăng tốc độ phát triển.

CHƯƠNG 6: KẾT LUẬN

6.1. Kết luận

Đồ án đã triển khai thành công hệ thống CI/CD hoàn chỉnh cho ứng dụng Mario Game trên Azure Kubernetes Service với hai môi trường Development và Production được tách biệt hoàn toàn.

Kiến thức đã học được và vận dụng:

- Terraform - Infrastructure as Code: Toàn bộ hạ tầng Azure được quản lý bằng Terraform với cấu trúc modules (network, AKS, ingress), cho phép tái tạo infrastructure nhanh chóng và nhất quán. Terraform state được version control, giúp tracking changes và rollback khi cần.
- Azure Cloud Platform: Sử dụng Azure Kubernetes Service, Virtual Network, Subnet, Load Balancer và Public IP. Hai môi trường được triển khai với Resource Groups riêng biệt và dải mạng tách biệt để đảm bảo isolation.
- Kubernetes & Container Orchestration: Quản lý containers với Deployments, Services, Ingress và Namespaces. Áp dụng resource limits, health checks (liveness/readiness probes) và multi-replica cho high availability trên Production.
- CI/CD với GitHub Actions: Pipeline tự động build Docker images, scan security với Trivy, push lên Docker Hub và update GitOps manifests. Development pipeline trigger tự động khi push code, Production pipeline sử dụng Git tags để control releases.
- GitOps với ArgoCD: Git là single source of truth cho application state. ArgoCD tự động sync changes từ Git vào cluster với auto-sync và self-heal, đảm bảo cluster luôn match với desired state.
- Helm Package Manager: Applications được đóng gói thành Helm charts với values files riêng cho từng môi trường, dễ dàng manage configuration và versioning.

6.2. Hạn chế

Hệ thống hiện tại còn một số hạn chế. Chi phí vận hành khá cao do hạ tầng Azure chạy 24/7, đặc biệt môi trường Production với hai node và Load Balancer, trong khi Dev chưa có auto-scaling hay lịch tắt ngoài giờ. Về bảo mật, chỉ áp dụng các biện pháp cơ bản như quét image và quản lý secrets, còn thiếu giám sát runtime, network policies, WAF, admission controllers và kiểm toán định kỳ.

Ứng dụng hiện đơn giản, chưa sử dụng database, nên các cơ chế StatefulSets, persistent volumes, backup/restore hay migrations chưa được triển khai. Hệ thống giám sát cũng chưa toàn diện, Prometheus và Grafana chưa đủ metrics, logging và alerting, gây khó khăn khi debug và theo dõi trạng thái ứng dụng. Pipeline chỉ thực hiện security scan, chưa tự động hóa unit tests, integration tests hay load tests, khiến QA chủ yếu làm thủ công.

Về High Availability, node chưa được phân bố đa Availability Zones, thiếu kế hoạch disaster recovery và triển khai đa vùng. Tối ưu chi phí chưa triệt để, chưa dùng Azure Reserved/Spot Instances và chưa điều chỉnh sizing tài nguyên theo nhu cầu thực tế.

6.3. Hướng phát triển

Trong tương lai, hệ thống có thể mở rộng để hỗ trợ ứng dụng phức tạp hơn, bao gồm database và dịch vụ stateful với StatefulSets, persistent volumes và cơ chế backup/restore. Pipeline CI/CD sẽ được nâng cấp với tự động hóa kiểm thử unit, integration và load tests, đồng thời tích hợp giám sát metrics, logging và alerting qua Prometheus/Grafana.

Về bảo mật, có thể áp dụng runtime security monitoring, network policies, WAF, admission controllers và audit định kỳ. Hạ tầng sẽ tối ưu bằng auto-scaling, triển khai đa zone và quản lý tài nguyên hiệu quả, giúp hệ thống mở rộng linh hoạt, duy trì sẵn sàng cao và vận hành ổn định hơn trong môi trường production.

TÀI LIỆU THAM KHẢO

Source code: <https://github.com/puppet1507/mario-aks-gitops>

Source code Mario: https://github.com/PlayMario/HTML5_Client

Terraform: <https://developer.hashicorp.com/terraform/docs>

ArgoCD: <https://argo-cd.readthedocs.io/en/latest/>

Helm: <https://helm.sh/docs/overview>

Github Actions: <https://docs.github.com/en/actions>

Deployment of Super Mario on Kubernetes using Terraform: [Aakib](#)